(https://profile.intra.42.fr)
(https://profile.intra.42.fr/searches)
gdeguign
(https://signin.intra.42.fr/users/sign_out)

Scale for project 42sh (/projects/42sh)

You should correct 4 students in this team

★

Git repository

vogsphere@vogsphere.42.fr:intra/2016/activities/42sh/cboussau

Introduction

Vous utilisez le SHELL depuis de nombreux mois, vous savez comment il doit réagir.

Soyez imaginatif sur les tests d'erreur et souvenez vous que trouver un SEGFAULT signifie pour les corrigés que leur travail n'a pas été assez minutieux. Pour le correcteur, cela signifie qu'il doit s'assurer de discuter de ce qui provoque ce SEGFAULT avec ses corrigés.

En cas de segfault, continuez néanmoins la soutenance pour échanger sur le sujet (sans appliquer le barême, puisque vous aurez bien entendu sélectionné l'onglet CRASH ci dessous).

Guidelines

Corrigez d'abord la partie obligatoire. Si et seulement si cette partie obligatoire est PARFAITE (complète et indestructible !), passez à la partie optionnelle. Gardez à l'esprit que le shell doit être intuitif, mais que les corrigés restent libres de l'implémentation de certains détails. Leur shell de référence n'est pas forcément le même que le votre.

La correction peut être longue si la partie optionnelle est bien fournie. Le 42sh n'est pas un projet trivial, prenez le temps qu'il faut pour examiner minutieusement le travail de vos camarades.

Bonne soutenance à tous !

Attachments

Subject (https://cdn.intra.42.fr/pdf/pdf/183/42sh.fr.pdf)

Sections

Préliminaires

Tests préliminaires

Vérifiez d'abord les éléments suivants :

- Il y a bien un rendu (dans le dépôt git)
- Fichier auteur valide
- Le Makefile est présent et compile bien l'exécutable 42sh
- Pas de faute de norme, la Norminette faisant foi
- Pas de triche (fonctions non autorisées, les étudiants doivent pouvoir expliquer leur code, ...)

Si un élément n'est pas conforme au sujet, la notation s'arrête la. Vous êtes encouragés à continuer de débattre du projet, mais le barème n'est pas appliqué.

☑ Yes ☒ No

La partie obligatoire

Acquisition minimale de ligne

- Affichage d'un prompt.
- Lecture de la ligne de commande, sans édition de ligne.
- Gestion correcte des espaces et des tabulations => pour vérifier tapez des espaces et/ou des tabs puis faites entrée, mettez des espaces/tabs entre les options d'une commande, etc.

☑ Yes ☒ No

Builtins basiques

Testez l'existence et le bon fonctionnement de toutes les builtins suivantes avec toutes leurs options si elles en ont :

- cd
- echo
- exit
- env
- setenv
- unsetenv

Pour chaque builtin, faites attention à bien tester toutes les options.

Testez également avec des arguments invalides (absents, mal formattés et mauvais nombre d'arguments).

Testez aussi les cas d'erreur, par exemple faites un cd sans argument (ce qui est équivalent à un "cd $HOME") après avoir unset la variable HOME de l'environnement. Soyez inventifs dans vos tests, le shell doit être robuste.

Ne cochez Oui que si TOUTES les builtins sont 100% fonctionnelles.

☑ Yes ☒ No

Exécution

Testez l'exécution de commandes simples avec leurs paramètres, et la gestion du PATH.

Vérifiez que les bons exécutables sont trouvés, dans l'ordre du PATH.
Faites également des tests avec des exécutables sur lesquels vous n'avez pas les droits d'exécution.

Le comportement du shell doit rester cohérent.

☑ Yes ☒ No

Erreurs et valeurs de retour

Testez la gestion des erreurs et de la valeur de retour des commandes.

Un "echo $?" report la valeur de retour des commandes ?

Testez par exemple une commande (et un builtin) avec des redirections d'entrée/sortie qui fail, ou des options inexistantes, et vérifiez que le retour de la commande n'est pas 0.

☑ Yes ☒ No

Quelques opérateurs de redirection

Testez l'existence et le bon fonctionnement des opérateurs de redirection suivants : ">", ">>", "<" et "|".

Chaînez en plusieurs, le comportement doit être cohérent.

Par exemple, que se passe-t-il si vous chaînez plusieurs redirections de sortie dans des fichiers différents ? ("ls > fichier1 > fichier2 > fichier3" typiquement) Est-ce que les étudiants peuvent justifier la manière dont leur shell gère ce type de cas ?

Testez des cas un peu particulier, comme l'utilisation d'un builtin dans une pipeline.

☑ Yes ☒ No

Opérateurs logiques

Testez l'existence et le bon fonctionnement des opérateurs logiques "&&" et "||".

Testez les avec des exemples complexes également, avec des pipelines par exemple.

☑ Yes ☒ No

Séparateur

Testez l'existence et le bon fonctionnement du séparateur ";".

☑ Yes ☒ No

La partie optionnelle

Rappel : vous ne devez évaluer la partie optionnelle que si la partie obligatoire est PARFAITE. Il n'y a aucun intérêt à développer des fonctionnalités exotiques sur un shell qui n'assure pas de manière parfaite les fonctionnalités de base !

Les inhibiteurs

Testez la presence et le bon fonctionnement des inhibiteurs """ (double quote), "'" (simple quote) et "\" (backslash).

Les simple quotes et les double quotes n'ont pas exactement le même comportement(*), vérifiez que les étudiants ont compris et implémenté ces différences.

*par exemple, les double quotes conservent l'échappement du backlsash "\" dans certains cas, ou encore elles doivent permettre l'évaluation de variables si le 42sh corrigé implémente les variables

☑ Yes ☒ No

Gestion du heredoc

Testez l'existence et le fonctionnement du heredoc "<<", avec ses options et cas particuliers.

☑ Yes ☒ No

Aggrégation des file descriptors

Testez l'existence et le fonctionnement de l'aggrégation des file descriptors.

N'oubliez pas de tester les cas d'erreur (file descriptor invalide ou récemment fermé) et l'argument "-" qui ferme le file descriptor spécifié avant l'opérateur de redirection.

☑ Yes ☒ No

Le globing

Testez la presence et le bon fonctionnement du globing ("*", "?", "[]", "{}", etc).

Regardez dans les sources l'implémentation du globbing, la fonction glob(3) ne doit pas être utilisée.

☑ Yes ☒ No

Yes No

Back quotes

Testez la presence et le bon fonctionnement de la substitution de commande avec les back quotes "^.

La commande exécutée doit l'être dans un sous-shell. Testez donc que l'environnement d'exécution du shell principal n'est pas modifié par l'exécution de la commande substituée (en lancant une commande qui modifie l'environnement, par exemple).

Yes No

Sous-shells

Testez la presence et le bon fonctionnement des sous shells ("()").

Pour cela, testez par exemple une builtin qui modifie l'environnement, et vérifiez que l'environnement n'est pas modifié dans le shell principal.

Testez également avec des commandes complexes.

Yes No

Variables locales

Testez la presence et le bon fonctionnement des variables locales et des builtins unset et export.

N'oubliez pas de tester les cas d'erreur (arguments invalides, non-existants, variable précédemment unset, etc).

Yes No

Historique

Testez la presence et le bon fonctionnement de l'historique des commandes et des builtins history et "!" avec toutes leurs options.

Yes No

Edition de ligne

Testez la presence et le bon fonctionnement de l'edition de ligne telle que demandée dans le ft_sh3.

Merci de vous référer au sujet du ft_sh3 pour le détail des fonctionnalitées attendues (partie obligatoire du ft_sh3 uniquement).

Les étudiants peuvent avoir choisi de mapper différemment les fonctionnalités aux touches clavier, ça n'a pas vraiment d'importance tant que le tout est cohérent.

Yes No

Manipulation de fichiers

Testez l'existence et le bon fonctionnement de la builtin read avec toutes ses options. (au minimum, l'option -r doit être présente)

Si le 42sh gère le shell script, pensez à vérifier le comportement de la builtin en mode non-intéractif.

Yes No

Complétion dynamique

Testez la presence et le bon fonctionnement de la complétion dynamique.

Au minimum, la complétion doit être fonctionnelle avec des fichiers et des dossiers, dans des sous-dossiers du répertoire courant.

Yes No

Job control

Testez la presence et le bon fonctionnement du Job control et des builtins job, fg, bg, (avec toutes leurs options !) et de l'operateur "&".

Comme d'habitude, testez d'abord toutes les options des builtins, en testant les cas d'erreur (arguments invalides, etc).

Testez également en envoyant divers signaux aux processus en background. Est-ce que leur status est correctement mis à jour dans le shell ?

Essayez de lancer une commande en background qui lit sur l'entrée standard. Le processus doit être interrompu par un signal généré par le TTY (un processus en background ne doit pas accéder au terminal de session). C'est bien le cas ?

Faites ensuite des tests avec quelques cas complexes, comme des pipelines ou les opérateurs logiques. Quel comportement a été choisi par les étudiants ? Est-ce que ce comportement est justifié et cohérent ?

Si le 42sh gère le shell script, pensez à vérifier le comportement de l'opérateur "&" avec un shell non-intéractif.

Yes No

Shell script

Testez la presence et le bon fonctionnement du shell script, c'est-à-dire l'implémentation d'un vrai langage de script shell dans le 42sh. Il ne s'agit pas de simplement "lire des fichiers avec des commandes" pour parler de shell script.

Testez d'abord que le parsing est sans failles : mettez des indentations et des sauts de ligne aléatoirement dans vos scripts de test (tout en respectant la syntaxe du langage shell, évidemment) et vérifiez que ça ne pose pas de problème.

Le shell script doit au minimum :
- implémenter les structures de contrôle (boucles, conditions)
- gérer les commandes en mode non-intéractif
- gérer des variables

Vérifiez méticuleusement ces points.

Yes No

Des bonus

D'autres fonctionnalitées

Si le 42sh a des fonctionnalitées supplémentaires, comptabilisez-les ici. Vous pouvez comptabiliser jusqu'à 5 fonctionnalitées bonus. Les bonus doivent être 100% fonctionnels et ne pas mettre en cause la stabilité du shell.

Rate it from 0 (failed) through 5 (excellent)

Ratings

Don't forget to check the flag corresponding to the defense

Ok Empty work Incomplete work No author file Invalid compilation Norme Cheat Crash

Conclusion

Leave a comment on this correction

*** (required) Comment**

Finish correction